

SW 품질을 위한 Agentic Engineering

Agenda

[SW 품질을 위한 Agentic Engineering]

- I. Coding Agent의 Engineering 발전 과정
- II. 비용 관점으로 바라본 Agentic Engineering

[Agentic Engineering 활용 전략]

- III. Agentic Engineering을 적용한 유지보수
- IV. 좋은 코드 판정하기
- V. 리팩토링 실습 (차량조립 KATA)
- VI. PoC와 Collection 수집
- VII. AE 관점으로 바라보기 (with SubAgent)

Chapter 01

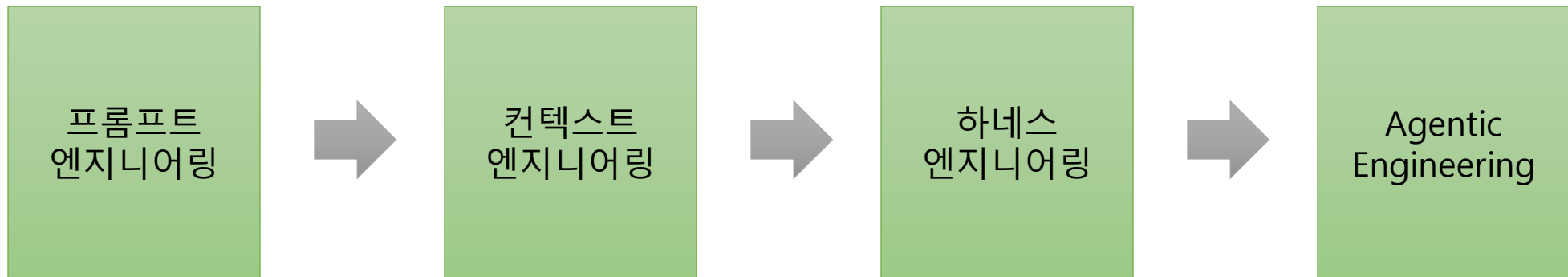
Coding Agent의 Engineering 발전 과정

SW 품질을 위한 Agentic Engineering

Coding Agent의 Engineering 발전 과정

이번 챕터에서는 Coding Agent의 활용과 함께 발전해 온 Engineering의 흐름을 살펴본다.

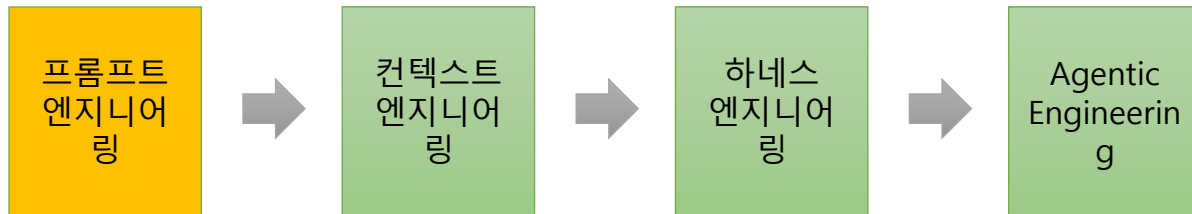
이 과정을 통해 각 엔지니어링 방법론에서 무엇이 중요했는지 이해하고,
나아가 Agentic Engineering에서는 무엇에 집중해야 하는지 파악할 수 있다.



Prompt Engineering (2020년)

2020년 당시 User -> LLM -> Answer 와 같이 입력을 보고 한번에 답을 생성하는 단순한 구조였으며, 당시에는 프롬프트 엔지니어링이 중요했음.

이 시기에는 결과의 품질이 대부분 프롬프트(입력 문장)에 의해 결정되었기 때문에, 어떻게 질문하고 어떤 맥락을 제공 하느냐에 초점을 맞추고, TIP들이 모여 Engineering이라는 이름으로 정리되었음



Prompt Engineering 예시

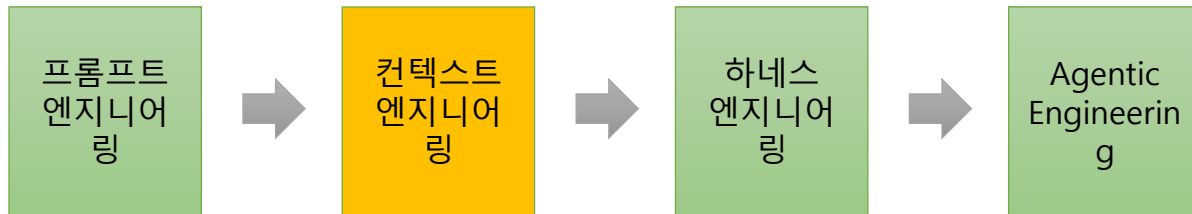
- Few-shot : 예제 몇 개 알려주면서 질문
- CoT : 단계별로 생각해줘 라는 키워드와 함께 질문
- Role 프롬프팅 : 역할 부여, 페르소나 라고 불리었음
- Output Format 지정 : 결과 형식을 요청
- Reflection : 답이 맞는지 한번 더 검토하라고 지시

Context Engineering (2023년)

LLM은 한 번에 처리할 수 있는 입력의 범위인 Context Window를 가진다.

이 제한된 범위 안에 어떤 정보를, 어떻게 구조적으로 넣을 것인가를 다루는 접근이 바로 Context Engineering이다.

핵심은 프롬프트에 포함될 정보 (Context Window)를 구조적으로 설계하는 방법이다.

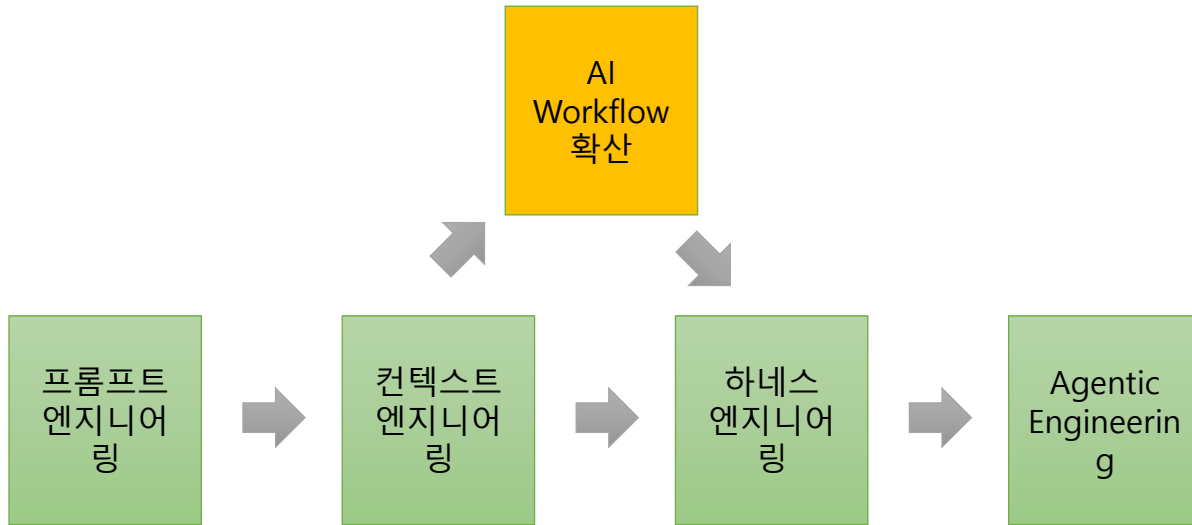


Context Engineering 예시

- 시스템 프롬프트 설계 (Rule 파일 등으로 구성)
- RAG, 메모리, 멀티턴 히스토리 구성
- 필요한 정보만 선별하여 효율적으로 전달하는 전략

AI Workflow 확산 (2023년)

한 번의 질문에 한 번의 답을 생성하는 1회성 LLM 동작만으로는 실제 AI 서비스를 구현하기 어려웠다. 단일 LLM 호출이 아니라, 여러 단계에 걸쳐 LLM을 반복적으로 활용하여 최종 결과를 만들어낸다. 결국, 실제 AI 서비스는 단일 동작이 아닌 Step 단위로 구성된 Workflow가 필요하다는 것이 명확해졌고, 이러한 흐름을 설계하고 관리하기 위한 도구로 LangGraph와 같은 Workflow 기반 라이브러리들이 주목받기 시작했다.



Agent로 구성되는 Workflow 패턴 양상

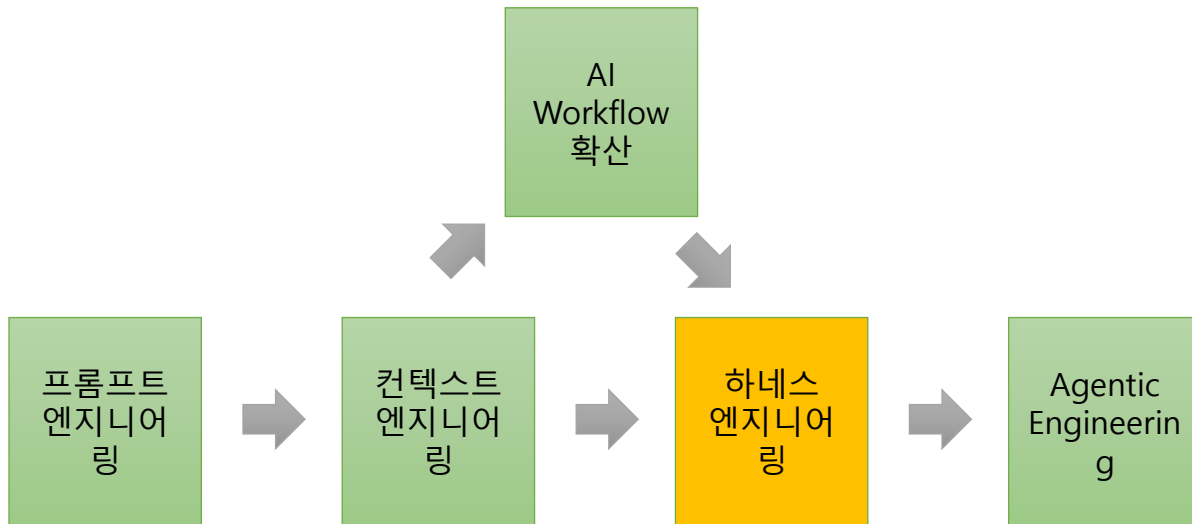
- 순차적(Sequential) Workflow
- 병렬적(Parallel) Workflow
- 평가-개선(Evaluator-optimizer) workflow

Harness Engineering (2025년)

AI Agent에서 “하네스(Harness)”는 Agent가 장시간으로 실행되더라도 올바른 임무를 수행하도록 하는 방법론이다.

에이전트 자체의 성능을 높이는 것보다, Workflow 레벨에서 안전장치를 설계하는 것을 하네스 엔지니어링이라 한다.

여기서 중요한 키워드는 **장시간 실행**이다.



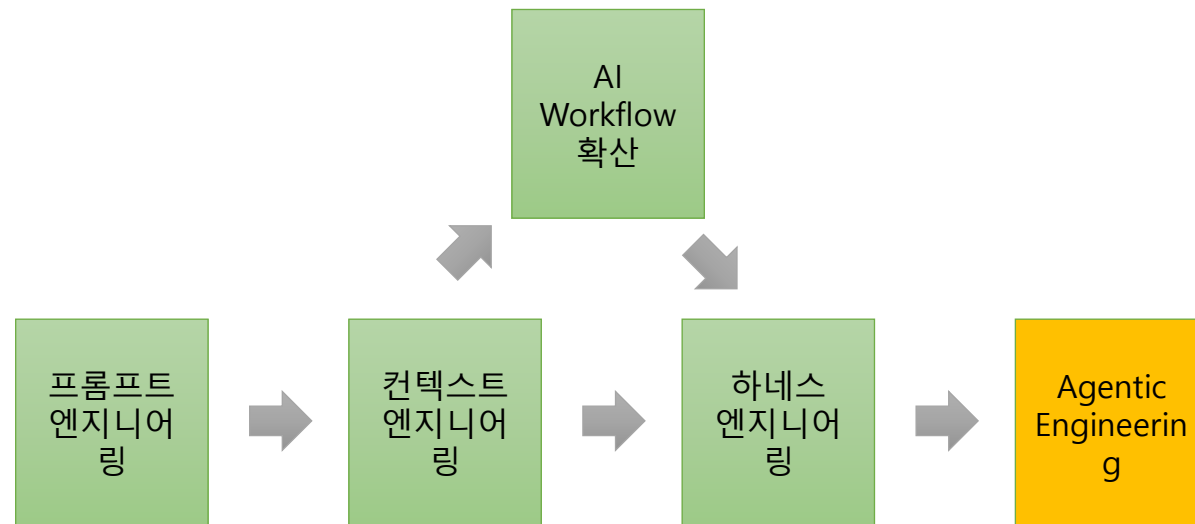
하네스 전 / 후

Agentic Engineering (2026)

Coding Agent로 인해 생산성은 높아지고, 기능도 잘 동작하지만,

SW 품질 저하로 고객에게 납품하기 힘든 제품이 만들어지는 문제 발생

1. 코드에 대한 이해에 대한 필요성이 적어지기 때문에 의도에 맞지 않는 숨어 있는 버그, 오버엔지니어링이 발생
2. 전체 코드 구조에 대한 이해도가 떨어져, 성능 최적화 포인트나 개선 아이디어를 도출하기 어려워 짐
3. 결과적으로 제품 SW 품질에 대한 신뢰와 자신감이 없음



Chapter 02

비용 관점으로 바라본 Agentic Engineering

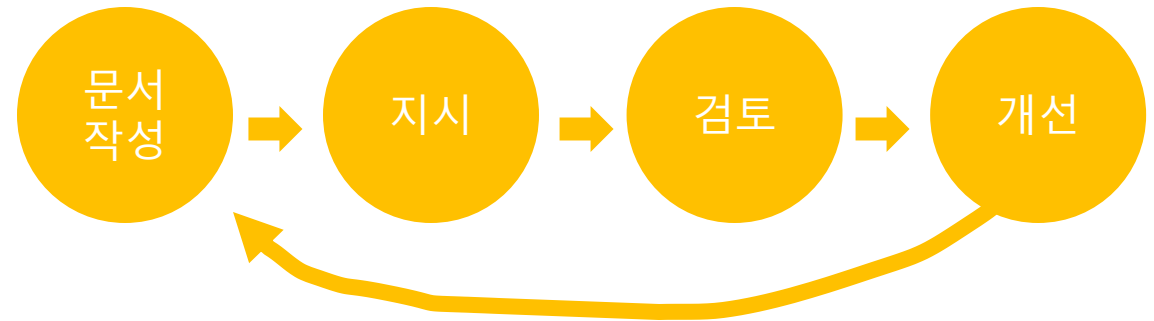
SW 품질을 위한 Agentic Engineering

ROI 관점에서 본 Agentic Engineering - 1

Agentic Engineering의 행동은 다음과 같다.

다음 작업을 위한 문서 작성

문서를 검토하고, 수정, 개선한다.
AI와 토론하며 내용을 보완한다.



지시하기

문서 내용을 기반으로 수행하도록 지시한다.

결과물 확인하기

결과물에 이해가 가지 않는 내용은 AI와 토론
문서가 구체적이지 않아 결과가 잘못 되었다면 문서부터 수정 후, 코드 변경을 지시한다.
마음에 드는 코드가 될 때까지 AI와 협업 하여 수정한다.

ROI 관점에서 본 Agentic Engineering - 2

Agentic Engineering에서의 비용은 크게 세 가지로 나눌 수 있다.

1. 플랜 문서 작성 및 검토 비용 : 코드 수정 비용 대비, 투자 효율이 좋다.
2. 실제 AI가 구현하는 비용 : 낮다.
3. 코드 리뷰 비용 : 매우 크다.

Agentic Engineering에서 ROI를 높이는 방법 - 1

Agentic Engineering에서의 ROI는 결국 리뷰 비용을 얼마나 줄이느냐에 달려 있다.

이를 위해서는 가독성과 분해 능력을 중심으로 설계해야 한다.

1) 가독성 확보 (클린코드)이 좋을수록 리뷰 비용은 줄어든다.

- 가독성이 좋을수록
 - 코드 이해 시간이 줄어들고,
 - 리뷰 속도가 빨라지며,
 - 전체 개발 시간이 단축된다.
- 클린코드는 곧 비용 절감이다.

Agentic Engineering에서 ROI를 높이는 방법 - 2

2) 리뷰 가능한 단위로 분해 (Phase 설계)

- 코드를 한 번에 크게 생성하는 것이 아니라, 사람이 충분히 검토할 수 있는 단위로 나누어 개발해야 한다.
- “리뷰 가능한 크기”로 나누는 것이 핵심이다.

3) 개발 역량 강화

- 개발자의 역량은 Agentic Engineering의 효율을 크게 좌우한다.
- 클린코드에 대한 고민과 리팩토링에 대한 경험이 많을수록, 일관된 가독성을 가진 코드를 설계할 수 있다.
- 개발 경험이 풍부할수록, AI가 생성할 코드의 형태를 예측할 수 있고, 리뷰 가능한 수준으로 Phase를 나누어 계획할 수 있다.
- 자료구조 / 알고리즘 이해도가 높을수록, 문제 분해 능력이 향상되며 AI가 사용할 구현 방식까지 예측할 수 있다.

Agentic Engineering

활용 전략

교육 내용 및 학습 목표

- 유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해한다.
- AI가 만든 코드를 바라볼 때, '좋은 코드'가 무엇인지 고민해보고, 리팩토링 KATA로 실습을 해 본다.
- 이후 PoC 수집에 대해 알아보고, Coding Agent 기능 중 SubAgent를 Agentic Engineering 관점에서 살펴본다.

Chapter 06

Agentic Engineering을 적용한 유지보수

SW 품질을 위한 Agentic Engineering

공학적인 관점의 전환

유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해해보자.

먼저 유지보수는 다음과 같은 목적으로 분류할 수 있다.

1. 기능 추가
2. 버그 수정
3. 버전 업데이트(마이그레이션)
4. 성능 개선

이 중에서도 기능 추가와 버그 수정이 가장 빈번하게 발생한다.

이번 챕터에서는 유지보수의 주요 활동인 기능 추가와 버그 수정을 중심으로, Agentic Engineering을 적용하는 방법을 살펴본다.

Agentic Engineering을 적용한 기능 추가

기능 추가 업무를 AI에게 맡기기 전에는, AI가 요구사항을 정확히 이해했는지 반드시 검증해야 한다.

또한 AI가 수행할 동작이 충분히 예측 가능한 수준까지 사전 논의가 이루어진 이후에, 개발을 지시해야 한다.

Agentic Engineering - 기능 추가 절차

1. Branch 생성 (Sandbox 구성)
2. 요구사항 정리 (AI와 함께 명확화)
3. 설계 문서 작성 (AI 활용 및 인간과 토론)
4. 설계 및 요구사항 검토 (인간 Review)
5. 개발 지시 및 구현 진행
6. 코드 리뷰 수행
7. Merge 수행

Agentic Engineering을 적용한 버그 수정

- 버그 수정은 단순히 코드를 고치는 작업이 아니라, 문제의 원인을 정확히 밝혀내는 과정이다.
- AI에게 바로 수정을 맡기는 방식은 증상만 제거하고, 근본 원인을 놓치는 결과를 초래할 수 있다.
- 이 근본 원인은 AI만 알면 안되고, 인간이 충분히 상황을 이해해야 한다.
- 만약 이해가 가지 않는다면, 수정작업 전, 끝까지 원인을 파헤쳐 알아야 한다.
그래야 예상치 못한 추가 버그가 일어나지 않는다.

Agentic Engineering - 버그수정 절차

1. Branch 생성 (Sandbox 구성)
2. 원인 분석 (AI와 함께 토론으로 명확화)
3. 재현 Scenario 및 재현 Test Case 확보
4. 수정 방향 검토 (AI가 작성, 인간의 검토)
5. 버그 수정 진행
6. 코드 리뷰 수행 & 재현 Test Case Pass 확인
7. Merge 수행

Chapter 07

좋은 코드 판정하기

SW 품질을 위한 Agentic Engineering

좋은 코드는 무엇일까?

- Agentic Engineering은 AI의 결과물을 인간이 검토하고 책임지는 개발 방식이다.
- 이번 챕터에서는 AI가 생성한 코드를 사람이 검토할 때,
어떤 관점으로 코드를 평가해야 하는지를 가볍게 살펴본다.

로버트 C 마틴의 클린코드

- 읽기 쉬운 코드가 좋은 코드이다. 가독성이 최우선이다.
- 이해하기 쉬운 코드는 곧 유지보수가 좋은 코드이다.
- 이름에는 의미가 있어야 하며, 함수는 한가지 역할을 담당해야 한다.
- 코드도 글처럼 읽히도록 해야 하며, 하나의 문서처럼 취급해야 한다.
- 클래스는 하나의 책임을 갖도록 한다. (SRP)
- 테스트코드는 필수이며, 자주 발생하는 중복은 제거해야 한다.

AI시대에 클린코드(가독성이 좋은 코드)는 어떤 장점이 있을까?

AI 코드리뷰 품질 상승

AI가 만든 코드를 사람이 검토해야 하는데, 사람의 검토가 정확하다.
사람과 AI의 협업 속도가 빨라진다.

AI 성능이 올라간다.

가독성이 좋은 코드는 복잡도가 낮다. 복잡도가 낮은 코드는 AI가 실수할 확률이 줄어든다.
(Hallucination 감소)
AI는 복잡도 낮은 코드에 대해 더 정확한 수정, 예상할 수 있는 방법으로 확장, 의도에 맞는 코드를 생성한다.

클린코드를 만들기 위해서는?

AI가 생성한 코드를 리뷰하다 보면, 코드가 읽기 어렵거나 복잡하게 느껴지는 경우가 있다. 이처럼 가독성이 떨어지거나 구조적인 문제가 있다고 판단될 때, 리팩토링을 수행해야 한다.

Rule of Three (3의 법칙)을 사용하자.

성급한 판단으로 추상화를 위한 리팩토링 진행 시, 코드 복잡도가 더 올라가기에 3회째 보이게 될 때 그때 리팩토링을 진행하자는 법칙이다.

- 1번째 관찰 : 그냥 코드 작성
- 2번째 관찰 : 중복이라고 생각이 들지만 지켜보고 넘어간다.
- 3번째 관찰 : 한번 더 보이면 리팩토링을 진행한다.

리팩토링은 그 범위에 따라 접근 방식이 달라질 수 있다.

작은 리팩토링(변수명 변경, 함수 분리 등)은 즉시 수행한다.

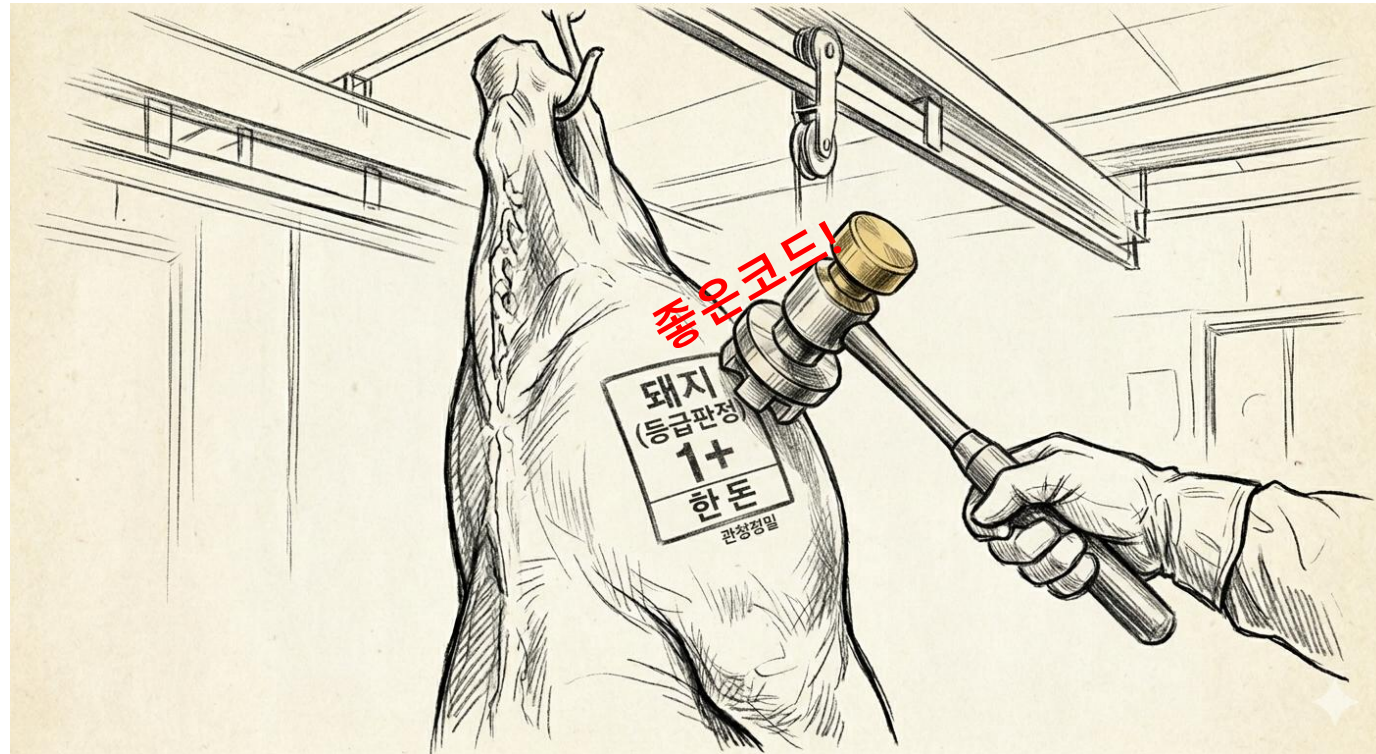
Class Level의 구조적 리팩토링이 필요한 경우에는, Agentic Engineering을 적용하여 별도의 브랜치를 생성하고 리팩토링 방향성에 대해 충분히 논의한 후 수행 및 리뷰를 진행한다.

사이먼윌리슨의 좋은코드

- 정확하게 동작한다 (버그 없이 의도대로 실행)
- 검증 가능하다 (테스트 등으로 올바름을 확인할 수 있음)
- 올바른 문제를 해결한다 (문제 정의에 부합)
- 예측 가능하게 실패한다 (에러 시 원인 파악 가능)
- 단순하고 이해 가능하다 (과도한 복잡성 없음)
- 테스트로 보호된다 (회귀 방지)
- 문서와 일치한다 (코드 변경 시 문서도 함께 갱신)
- 변경에 유연하다 (YAGNI 준수 + 변화 대응 가능)
- 비기능 품질을 갖춘다 (성능, 보안, 유지보수성, 확장성, 관찰 가능성 등)

좋은 코드 판정하기

- AI가 생성한 코드는 반드시 사람이 직접 코드 리뷰를 수행해야 한다.
- 리뷰가 완료되면, 해당 코드가 검증되었음을 나타내는 명시적인 기록을 남겨야 한다.
- 이 기록은 Commit 시 Commit Message를 통해 표현할 수 있으며, 팀에서 정의한 Commit Message Template을 활용하여 "좋은 코드 판정 마크" 의미의 기록을 남기는 것이 바람직하다.



리뷰는 책임이 따라야 한다.

- 리뷰는 책임을 가져야 한다. AI 시대 이전과 그 책임의 무게는 완전히 다르다.
- AI 시대에서 코드 리뷰는 단순한 확인 절차가 아니라, 결과에 대한 책임을 지겠다는 행위이어야 한다.
- 과거에는 개발자가 직접 코드를 작성했지만, 이제는 AI가 초안을 생성하고, 인간이 최종 검증자 역할을 수행한다.
- 따라서 개발자는 단순히 코드를 읽고 넘어가는 것이 아니라,
- 이 코드를 충분히 검토했으며, 문제가 발생할 경우 그 책임을 내가 진다 라는 의미의 판단을 기록으로 남겨야 한다.

AI가 코드를 작성하는 시대에는,
코드를 만든 주체(AI)가 아니라 승인한 사람이 책임을 진다.

[도전] 리뷰 도장 만들기

코드 책임 기반 리뷰 정책 예시

- 코드를 생성하여 Commit하는 사람은 그 코드에 책임자이며, 잘못되는 경우 불이익을 받을 수 있음을 이해한다.
- Commit에 "리뷰 도장" 내용을 남긴다.
- 코드에 문제가 발생했을 때, Commit을 수행했던 사람이 책임을 지고, 끝까지 원인을 파헤친다. 이러한 이력은 기록을 남겨야 한다.
- 리뷰 실패로 인한 이력은 누적 관리되고, 부서장은 누적된 리뷰실패 이력을 모니터링 할 수 있어야 한다.

[도전]

- Commit 메시지에 어떻게 "리뷰 도장"의 의미를 갖는
- Template을 가져갈지 샘플을 각자 만들어보자.



Chapter 08

리팩토링 실습 (Gilded Rose KATA)

SW 품질을 위한 Agentic Engineering

Gilded Rose Requirements Specification

Gilded Rose 는 Allison 이 운영하는 작은 여관입니다. 이 여관에서 최고급 상품만을 사고 팝니다. 하지만 아쉽게도, 우리의 상품은 **판매 기한**이 지날수록 **품질**이 점점 떨어집니다. 이런 상황에 맞게 자동으로 재고의 정보들을 갱신해주는 시스템이 있습니다.

현재 시스템의 기본 규칙

- 모든 아이템은 **SellIn** 값을 갖고 있으며, 이는 **판매까지 남은 일수**를 뜻합니다. (SellIn = 판매가능기한)
- 모든 아이템은 **Quality** 값을 갖고 있으며, 이는 **아이템의 가치**를 나타냅니다. (Quality = 아이템 가치)
- 하루가 끝나면 시스템은 모든 아이템의 **SellIn** 과 **Quality** 값을 변경합니다

현재 시스템의 세부규칙

- 판매기한이 지나면, **Quality** 는 **두 배 빠르게** 떨어집니다.
- Quality 는 절대 0 미만이 될 수 없습니다.
- Quality 는 절대 50 을 초과할 수 없습니다.
- "Aged Brie" 는 시간이 지날수록 **Quality** 가 상승합니다.
- "Sulfuras" 는 전설의 아이템으로, 판매할 필요도 없고 **Quality** 도 변하지 않습니다.
- "Backstage passes" 는 판매기한이 다가올수록 **Quality** 가 증가합니다.
 - 10일 이하일 때 Quality는 2씩 증가
 - 5일 이하일 때는 Quality는 3씩 증가
 - 콘서트가 끝나는 판매 불가 시점이 되면 Quality는 0이 됩니다.



소스코드 링크

- <https://github.com/mincoding1/GildedRose>



게임 WOW의 상점(여관)인 Gilded Rose



전설의 아이템 "Sulfuras, Hand of Ragnaros"

요구사항 명세

상품들을 판매하는 Lagacy 시스템이 있다.

우리 상점의 상품들은 판매 기한이 가까워질수록
가격이 변동된다.



Mission Goal

상품들을 관리하는 Legacy 시스템이 있고,
해당 시스템에서는 아이템들의 SellIn, Quality 를 변경합니다.

- 현재 Legacy 시스템의 코드는 너무 난잡한 상태로 디자인 변경을 위해서는 리팩토링이 필요합니다.
- 현재 Legacy 시스템에는 테스트 코드가 없는 상태입니다. 리팩토링을 위해 테스트코드 추가가 필요합니다.
- 시스템에 새로운 상품(e.g. Conjured 아이템 추가)을 추가할 예정입니다. 기능 추가를 하기 쉽도록 디자인 변경을 해주세요.

SellIn 과 Quality

모든 아이템은 SellIn 값과 Quality 값을 가집니다.

SellIn

판매 가능한 기간이고 하루에 1씩 줄어듭니다.

Quality

상품의 가치입니다.

시간이 지날수록 가치가 줄어드는 아이템도 있고
시간이 지날수록 가치가 올라가는 아이템도 있고
시간이 지나도 가치가 유지되는 아이템이 있습니다.

판매 상품 정보

상품을 총 네 가지로 분류하여 판매 중이고,
상품을 더 추가해야 함을 염두에 두고 리팩토링을 한다

일반
아이템

<else>

전설의
아이템

Sulfras...

숙성된
치즈

Aged Brie

콘서트
입장권

Backstage
passes ...

마법
아이템

Conjured

상품 추가 개발해야함
(추가 기능 구현 내용이지만,
우리 수업에서 다루지 않음)

Quality 변경

각 아이템들은 하루가 지날 때 마다 Quality가 변동되며,
Quality 의 변경 규칙은 상품별로 다르다.

일반
아이템

<else>

값 : 0 ~ 50
매일 1씩 떨어짐
최소 0원

전설의
아이템

Sulfras...

값 : 80
매일 가격 변동 없음

숙성된
치즈

Aged Brie

값 : 0 ~ 50
매일 1씩 가격이 오름
최대 50원

콘서트
입장권

Backstage
passes ...

값 : 0 ~ 50
콘서트가 가까워지면
가격이 비싸짐
최대 50원

마법
아이템

Conjured

값 : 0 ~ 50
일반 아이템에 2배씩 가격 떨어짐
(추가 기능 구현 내용이지만,
우리는 신경쓰지 않는다.)

아이템별 Quality 변화

일반아이템 (Quality 범위 0 ~ 50)

하루에 가격 1씩 감소

남은 판매기한이 음수가 될 때 부터 가격 2씩 감소

숙성된 치즈 (Quality 범위 0 ~ 50)

하루에 가치가 1씩 증가

남은 판매기한이 음수가 될 때 부터 가치가 2씩 증가

콘서트 입장권 (Quality 범위 0 ~ 50)

판매 가능 기한에 따라 가치 변동이 다르다.

판매가능기한이 10일 보다 크면 가치가 1씩 증가

6 ~ 10일에 하루가 지나면 가치가 2씩 증가

1 ~ 5일에 하루가 지나면 가치가 3씩 증가

남은 판매기한이 음수가 될 때 부터 상품의 가치는 무조건 0 (콘서트 종료 되었기 때문)

전설아이템

하루가 지나도 판매 가능 기한과 상품의 가치는 변경되지 않음

소스코드 구조

클래스

GildedRose : 상점 아이템 관리 시스템

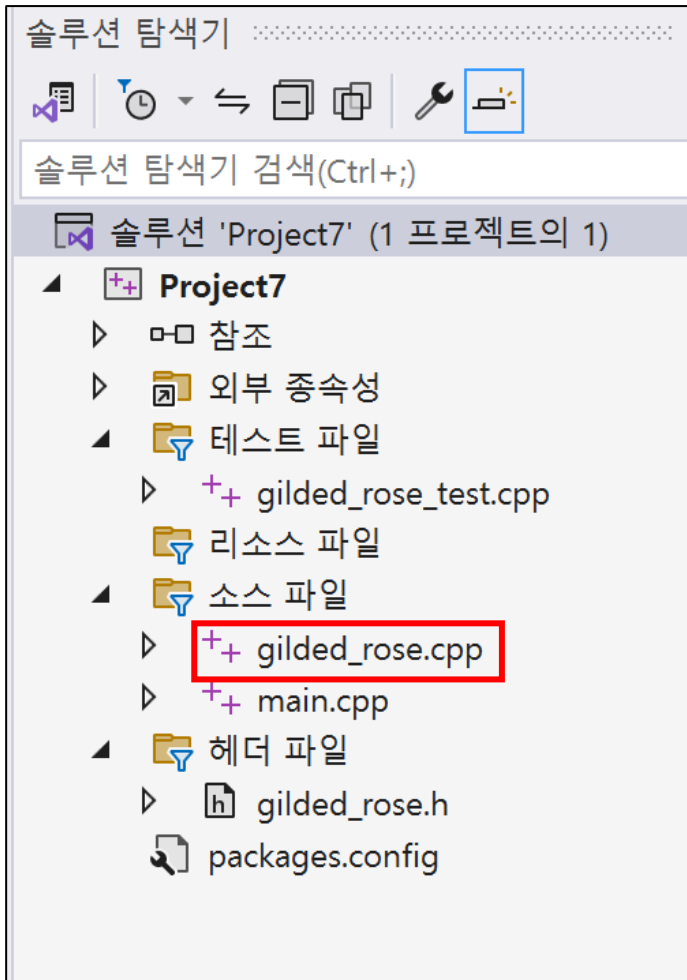
Item : Item의 정보를 갖는 class

하루가 지날 때마다 값을 갱신하는 메서드

Gilded Rose class의 update_quality() 메서드

gilded_rose.cpp의 **update_quality** 메서드를 가볍게 분석해본다.

무조건적인 리팩토링이 아닌, 이해 한 것을 바탕으로 리팩토링을 해야한다.



```
#include "gilded_rose.h"

using std::vector;
using std::string;

GildedRose::GildedRose(vector<Item>& items) : items(items)
{
}

void GildedRose::updateQuality()
{
    for (int i = 0; i < items.size(); i++)
    {
        if (items[i].name != "Aged Brie" && items[i].name != "Backstage passes to a TAFKAL80ETC concert")
        {
            if (items[i].quality > 0)
            {
                if (items[i].name != "Sulfuras, Hand of Ragnaros")
                {
                    items[i].quality = items[i].quality - 1;
                }
            }
        }
        else
        {

```

updateQuality() 함수가 리팩토링 대상이다.

리팩토링 준비

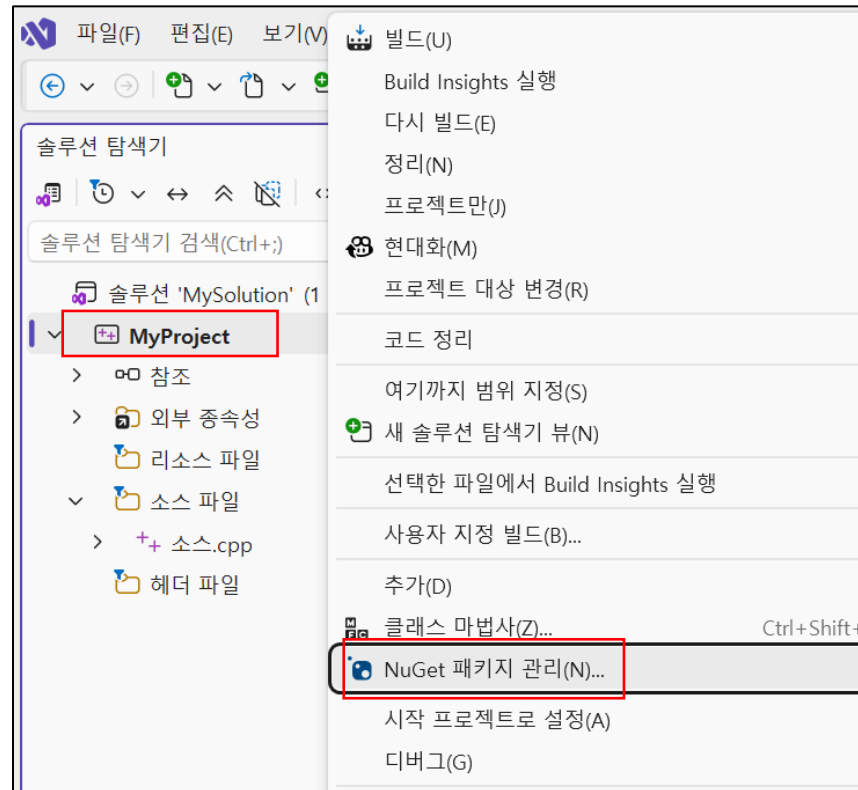
테스트 준비

리팩토링 전, 테스트 환경을 준비한다.

리팩토링 대상이 되는 코드는 커버리지 100% 가 이상적이다.

Google Test 설치 방법 - 1

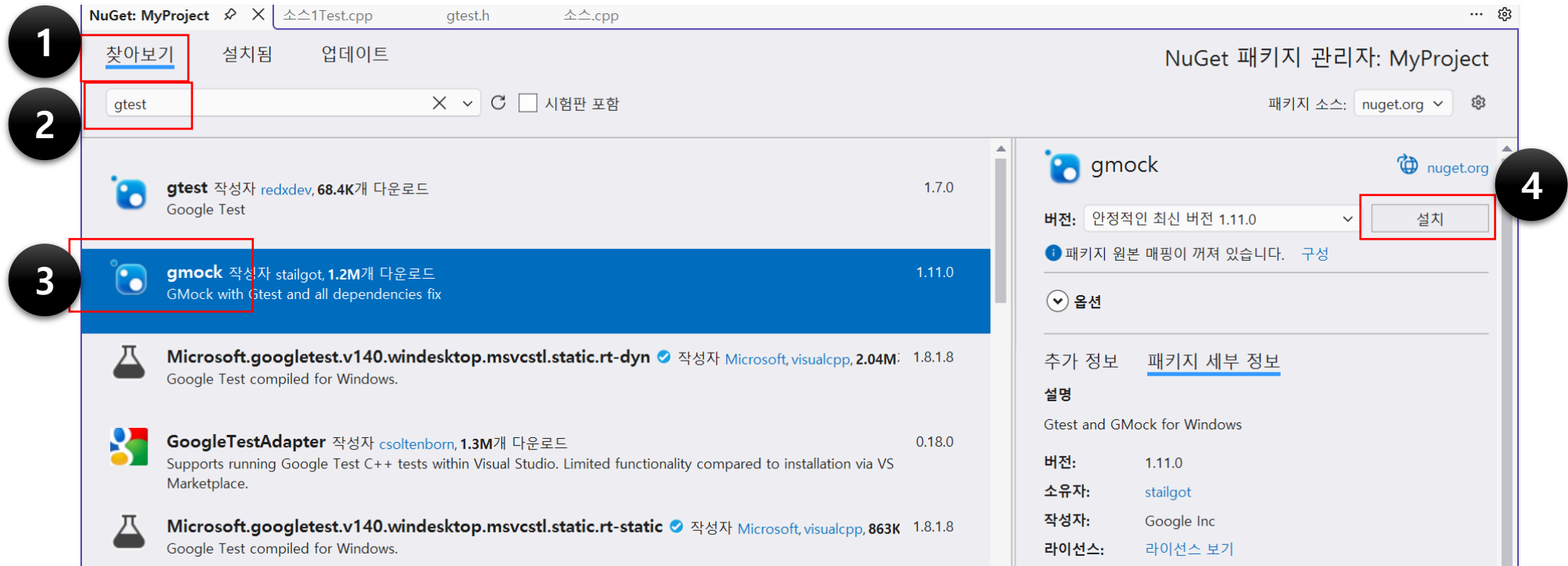
프로젝트 > NuGet 패키지관리



Google Test 설치 방법 - 2

NuGet 패키지 관리자

MS 개발환경에서 Library 를 쉽게 설치할 수 있도록 만들어진 도구
gmock 을 설치한다. (5초 소요)

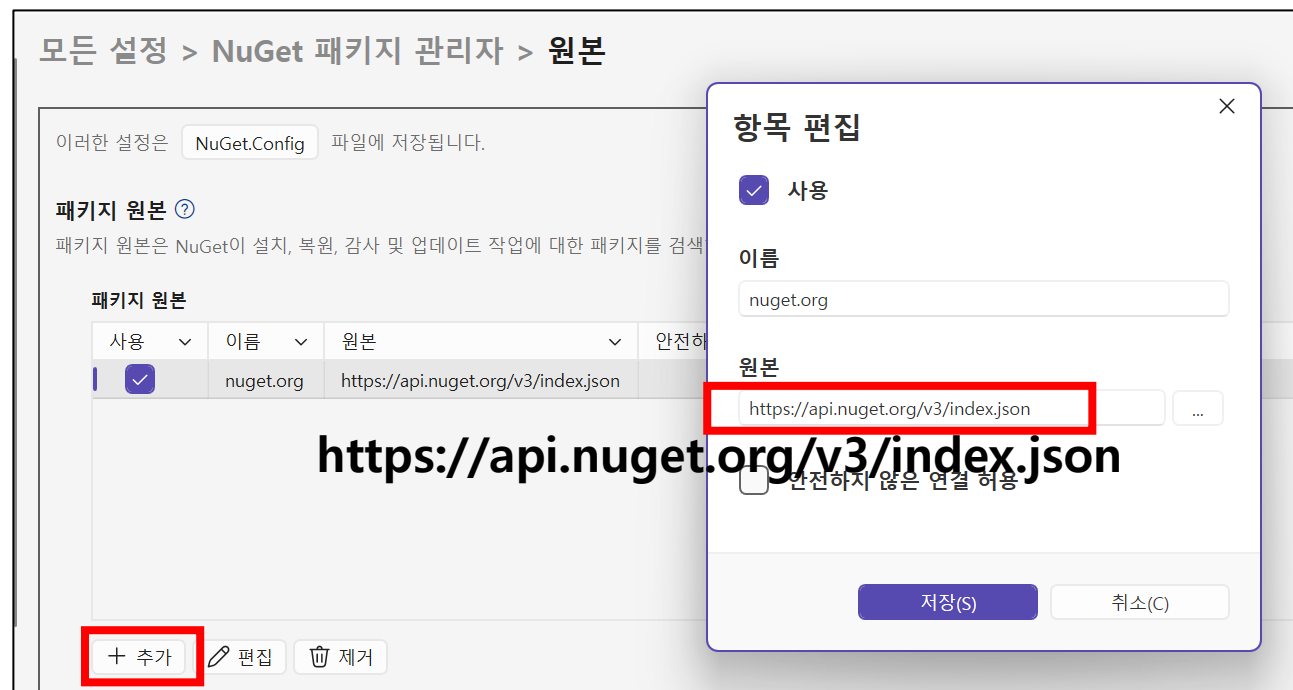
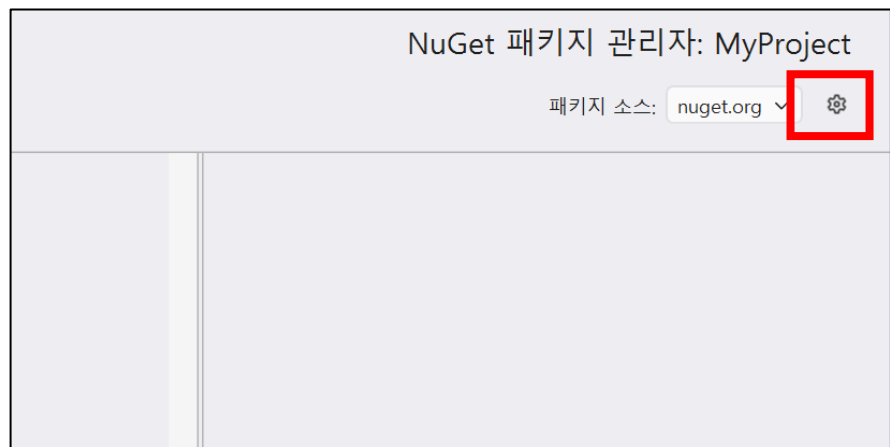


[Trouble Shooting] NuGet 저장소 확인

Nuget에 검색이 안되는 경우

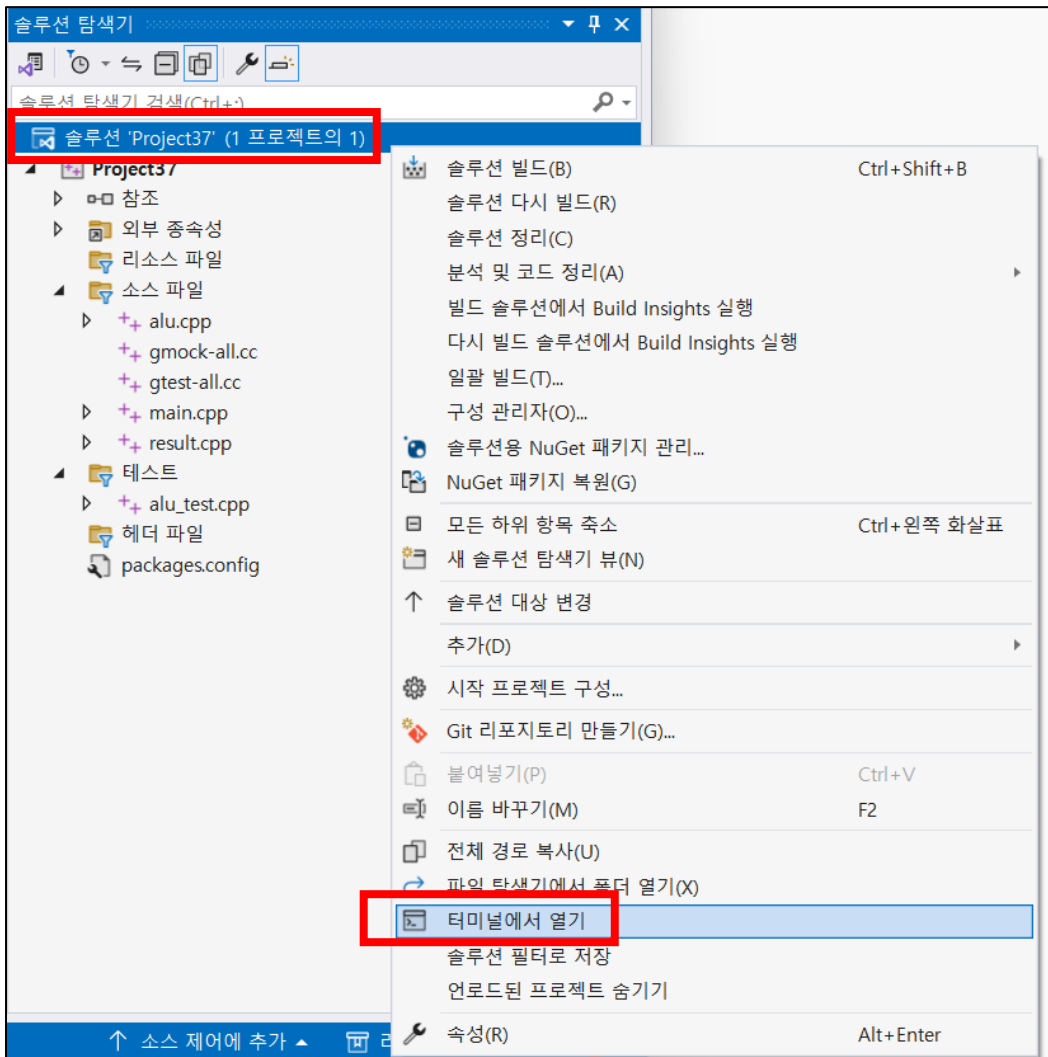
현재 다음과 같이 패키지 소스 경로가 맞는지 확인한다.

다르다면 아래와 같이 수정한다.



[참고] 코드 커버리지 측정 방법 1

프로젝트가 아닌 “솔루션” 에서 터미널 열기



```
개발자 PowerShell
+ 개발자 PowerShell
*****
** Visual Studio 2022 Developer PowerShell v17.11.5
** Copyright (c) 2022 Microsoft Corporation
*****
PS C:\Users\user\source\repos\Project37>
```

[참고] 코드 커버리지 측정 방법 2

`OpenCppCoverage.exe` --sources C:*.cpp --export_type=html:coverage -- .\x64\Debug\실행파일이름.exe

복사 붙여넣기를 위한 명령어 링크

<https://gist.github.com/mincoding1/3d187a62a3ea90c5c21aa68016c408b6>

```
개발자 PowerShell
+ 개발자 PowerShell |
+
*****
** Visual Studio 2022 Developer PowerShell v17.11.5
** Copyright (c) 2022 Microsoft Corporation
*****
PS C:\Users\user\source\repos\Project37> OpenCppCoverage.exe --sources C:*.cpp --export_type=html:coverage -- .\x64\Debug\
```

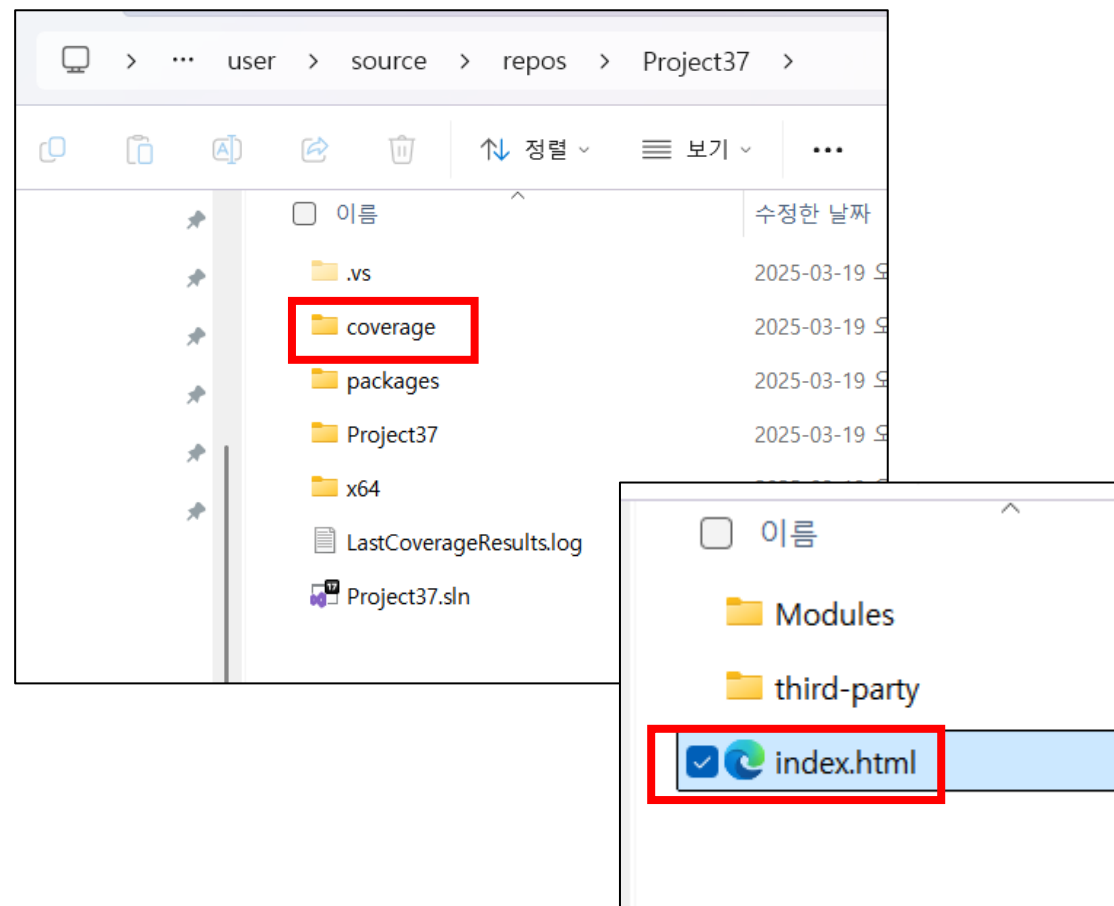
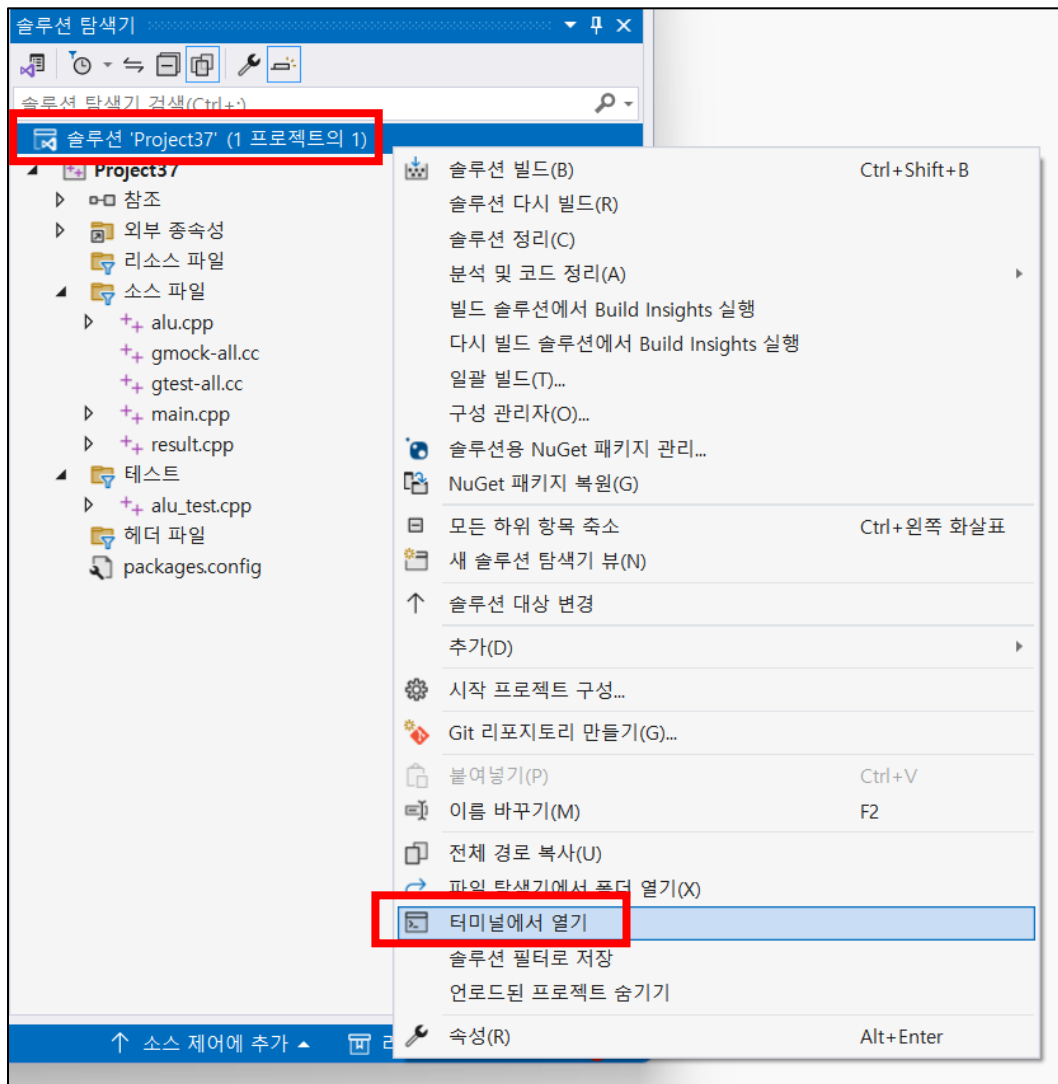


탭키 여러번 눌러
.exe파일이 나오도록하기

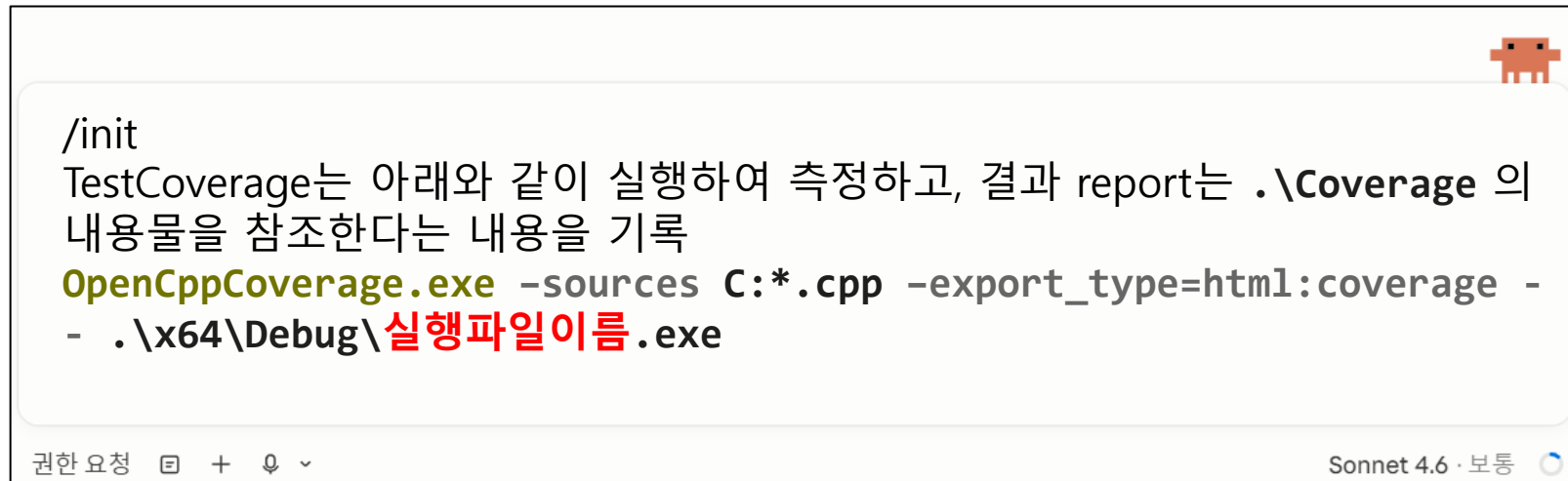
```
개발자 PowerShell
+ 개발자 PowerShell |
+
*****
** Visual Studio 2022 Developer PowerShell v17.11.5
** Copyright (c) 2022 Microsoft Corporation
*****
PS C:\Users\user\source\repos\Project37> OpenCppCoverage.exe --sources C:*.cpp --export_type=html:coverage -- .\x64\Debug\Project37.exe
```

[참고] 코드 커버리지 측정 방법 3

커버리지 결과 확인



본격적인 개발을 시작하기 전 CLAUDE.md를 만들어 기초 준비 Test Coverage 측정 방법과 Build 방법 명시



```
/init
TestCoverage는 아래와 같이 실행하여 측정하고, 결과 report는 .\Coverage 의
내용물을 참조한다는 내용을 기록
OpenCppCoverage.exe -sources C:*.cpp -export_type=html:coverage -
- .\x64\Debug\실행파일이름.exe
```

권한 요청  Sonnet 4.6 · 보통 

PLAN 작성

개발을 지시하기에 앞서 작은 Phase로
작업 절차를 나눠 PLAN 세우기



gilded_rose.cpp 을 refactoring할 예정
UnitTest부터 작성하고,
영향이 작은 변경점부터 점차 큰 변경의 형태로 진행
이를 위한 plan을 docs/plan.md에 작성

권한 요청

Sonnet 4.6 · 보통

Plan 을 기반으로 Phase 0 부터 Design 하기



/docs/plan.md의 각 Phase는

1. 해당 Phase의 Design문서 작성
2. 사람이 직접 해당 문서 검토
3. 해당 Phase를 Design 대로 진행
4. 사람이 직접 해당 코드 검토

의 순서로 작업해 나갈 예정

문서 연결

본격적인 리팩토링을 시작하기 전 CLAUDE.md에 문서 연결하여
이후의 흐름에서도 전체적인 문서 관리가 되도록 준비



사전 문서 작업 사항 Commit

작업 중 이슈가 발생하여 되돌아 올 수 있어야 하고,
Agentic Engineering의 절차대로 진행했다는 근거인 commit 남기기



Phase0 Design

Phase 0번에 대하여 Design 지시



Design된 문서를 검토 직접 후



Phase0 Commit

검토가 완료된 Phase 0에 대하여 Commit



이후 Phase

마찬가지 흐름으로 1~이후 Phase도 진행

진행시 Phase에 추가/수정 하고 싶은 내용의 경우

1. 직접 문서들을 수정
2. AI와 논의하여 AI가 수정

Chapter 09

PoC와 Collection 수집

SW 품질을 위한 Agentic Engineering

개발을 진행하다 보면 익숙하지 않은 기능이나, 사용은 해야 하지만 구현 방식이 예상되지 않는 기술을 써야 할 경우가 있다.

또한 해당 기술이 현재 프로젝트에 성능이 잘 나오는지 확인이 필요한 경우도 있다.

이러한 경우, 현재 프로젝트에 바로 적용하기보다 PoC(Proof of Concept, 탐색적 프로토타입)을 먼저 수행하는 것이 좋다.

PoC 접근 방식

1. 현재 프로젝트와 분리된 독립적인 환경에서 진행한다
2. 빠르게 검증하는 것이 목적이므로, Vibe Coding 방식으로 빠르게 구현한다.
3. 이 단계에서는 Agentic Engineering을 적용할 필요는 없다.

[도전] PoC 만들기

익숙하지 않은 코드를 직접 PoC 직접 해보고, 코드에 대해 이해해본다.

- Json 데이터 파싱 / Json 파일 저장하는 Library - PoC 해보기
- Quick Sort 를 모른다고 가정하고, 알고리즘 PoC 해보기

Collection - 좋은 프롬프트 / 코드를 모으자.

좋은 프롬프트를 모으자

- 내가 지시할 때 사용할 우수한 프롬프트 세트를 체계적으로 정리하고 Skill로 만들어도 좋다.
- 타인이 만든 좋은 프롬프트나 Skills를 사용할 때는 반드시 내용을 이해한 후에만 사용한다.
- Skills의 동작과 목적을 모른 채 사용하는 것은 금지한다.

좋은 예제 코드를 모으자.

내가 코드를 완벽히 이해한, 마음에 드는 좋은 코드들을 모아두자.

✓ PoC가 끝난 코드들만 모아두어도 좋다.

아래와 같은 방식으로, 지시할 수 있다.

[github.com/ ecosystem/datasette-oauth](https://github.com/ecosystem/datasette-oauth) 방식을 참고하여,
~/dev/ecosystem/llm-mistral 프로젝트에 모의 HTTP 테스트를 추가하세요.

프롬프트 / 소스코드를 모아둔 나만의 Collection을 만들자.

내가 검증하고 이해한 자료들만 선별하여 정리하자.

단순 수집이 아니라, 재사용 가능한 나만의 기준과 패턴으로 축적하자.

언제든 꺼내 쓸 수 있도록 구조화된 형태로 관리하자.

모아둘 때 사용하는 도구 예시

- GitHub / Gist
- Confluence / Notion / Obsidian

[도전] PoC 활용하여 Application 개발

기존에 진행했던 PoC(Proof of Concept) 코드의 구조와 로직을 참고하여,
데이터를 JSON 파일로 관리하는 CRUD(Create, Read, Update, Delete) 콘솔 애플리케이션을 개발한다.

요구사항

- PoC에서 사용된 코드 구조를 유지한 상태로, CRUD 구현

CRUD 기능 구현

- **Create:** 새로운 데이터를 입력 받아 JSON 파일에 저장.
- **Read:** 전체 목록 보기 및 특정 ID/키 값으로 검색 기능.
- **Update:** 기존 데이터를 선택하여 특정 필드 수정.
- **Delete:** 특정 데이터를 안전하게 삭제.

Chapter 10

AE 관점으로 바라보기 (with SubAgent)

SW 품질을 위한 Agentic Engineering

챕터의 목표

앞으로 다양한 Agent 기능들이 추가될 예정이다.

- SubAgent는 이미 잘 활용되고 있고,
- Agent가 사람에게 먼저 질문하는 기능
- Harness 기능들 등

이러한 기능들이 추가될수록,

각 기능을 Agent Engineering 관점에서 어떻게 사용할 지
팀에서 고민하고 결정해야 한다.

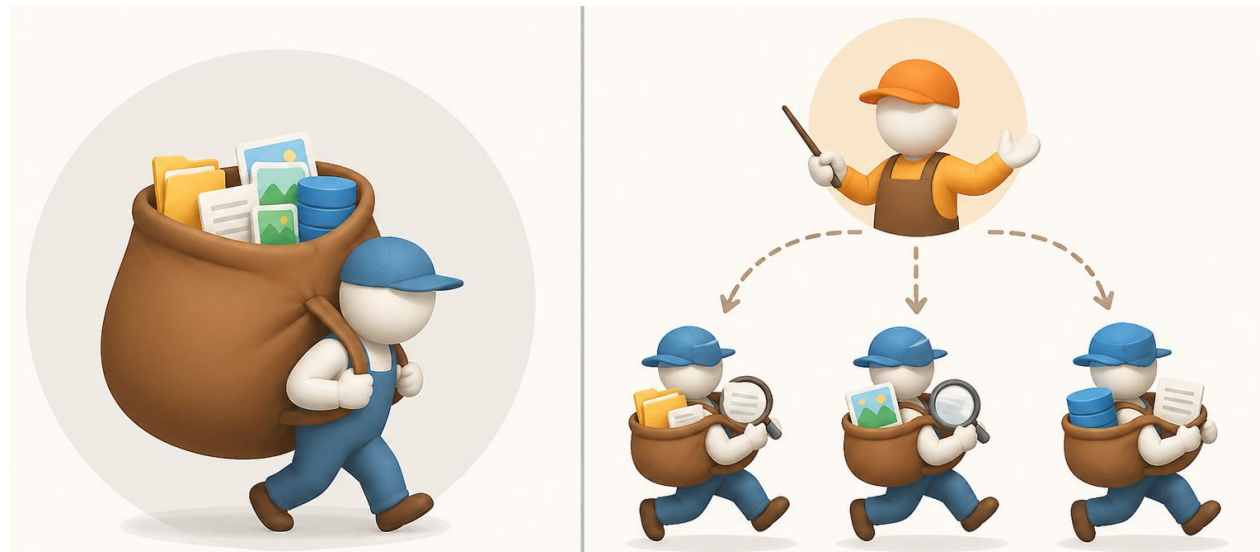
이번 챕터에서는 SubAgent 를 예시로 A.E 관점에서 바라볼 것이다.

[복습] Subagent

SubAgent는 하나의 큰 Agent 내부에서 특정 역할을 맡아 독립적으로 동작하는 “작은 Agent”이다.

- **예시 : Main Agent (Orchestrator)**
 - SubAgent1 : Plan 작성
 - SubAgent2 : 기존 코드 분석
 - SubAgent3 : 기존 코드 테스트

이는 AI 개발 실행 시간 줄이고자 하는 목적이 아님을 기억해야 한다. 각 Agent가 본인의 역할에 맞는 독립적인 Context Window로 가지게 되며, 이는 더 높은 정확도를 가진 결과물을 낼 수 있는 효과가 있다.



SubAgent의 단점

SubAgent 구조는 AI의 성능을 올릴 수 있지만, 그 만큼 사람의 검증이 더 어렵다.

단일 Agent 실행

하나의 흐름으로 결과가 나오기 때문에, 결과를 실시간으로 평가가 가능하다.

SubAgent 구조

SubAgent의 중간 결과에 대해 검토하기 어렵다.

특히 병렬로 수행되는 경우, 중간 검토도 하기 전에 최종 결과를 만들어낸다.

AE 관점에서 SubAgent 사용 방안

SubAgent를 활용하더라도, Agentic Engineering 원칙에 맞게 구조적으로 사용하는 것이 중요하다.

Plan 이후 Action 단계

- 전체 Plan을 먼저 수립한 뒤, 실제 실행(Action)은 SubAgent에게 위임한다.
- Plan만 명확하게 유지된다면 내부 실행 방식은 유연해도 좋다.
- 각 SubAgent는 자신의 역할에 맞는 독립적인 Context로 작업을 수행하므로 결과 정확도가 올라간다.

Human-in-the-loop 이전, Verify 단계

- Action 수행 이후, 사람의 개입 전에 중간 검증 단계를 둘 수 있다.
- 이때 SubAgent를 활용하면 검증 역할을 독립적으로 수행할 수 있다.
- 결과적으로 사람은 더 정제된 결과를 확인하게 되어 검토 효율이 올라간다.

PoC (탐색적 개발) 단계

- 중간 검토가 필수적이지 않은 상황이므로, SubAgent는 자유롭게 사용 가능하다.

감사합니다.